Homework 2: Policy Gradient vs Value-Based RL in JAX

ELG5214 / CSI5340 — Introduction to Deep Reinforcement Learning

Mohammad ., Student No: 300480272

University of Ottawa — Winter 2026

Abstract. This report presents a comparative study of Deep Q-Network (DQN) and REINFORCE on a custom 5×5 grid-world environment with obstacles and a trap, implemented entirely in JAX using the Gymnax framework. Both algorithms are benchmarked against a random baseline across 10 independent seeds and 6 hyperparameter configurations (3 learning rates × 2 discount factors) each combination running for 300 episodes. DQN with lr=0.01 and gamma=0.9 achieved perfect goal-reaching success (100%), while REINFORCE's best configuration (lr=0.1, gamma=0.99) reached 79% success, revealing fundamental differences in how value-based and policy-gradient methods handle sparse, structured reward landscapes.

1. Environment Design and Rationale

The environment is a custom 5×5 grid-world called **ObstacleTrapGridWorld**, built as a fully JAX-compatible Gymnax environment by subclassing `gymnax.environments.environment.Environment`. The agent starts at position (4, 0) — the bottom-left corner — and must navigate to the goal at (0, 4) — the top-right corner, for a minimum Manhattan distance of 8 steps.

1.1 Grid Layout

The grid contains three types of special cells beyond the start and goal: (i) three **impassable obstacles** at positions (1,1), (2,2), and (3,1) shown as black cells; (ii) a **trap** at position (1,3) shown in pink, which delivers a harsh penalty of -2.0 but does not terminate the episode; and (iii) boundary walls — moving into a wall or obstacle keeps the agent in place and incurs a penalty of -0.2.

The obstacles were placed along the diagonal to create a natural barrier that forces the agent to find a non-trivial path. The trap at (1,3) sits directly on the most tempting shortcut to the goal, testing whether the agent learns to avoid locally appealing but costly routes. This design creates a tension between exploration efficiency and reward maximisation that differentiates the two algorithmic families.

1.2 Reward Structure

Event	Reward	Rationale
Reach goal (0,4)	+10.0	Large positive signal for task completion
Step on trap (1,3)	-2.0	Harsh penalty; tests avoidance learning
Hit wall / obstacle	-0.2	Mild penalty discourages boundary bumping
Normal step	-0.05	Small cost encourages shorter paths

Table 1: Reward structure of the ObstacleTrapGridWorld.

The reward design is intentionally **semi-sparse**: the dominant positive signal (+10.0) is only received upon reaching the goal, while step penalties are small (-0.05). This creates a challenging credit assignment problem — the agent must connect a long sequence of actions to a delayed reward. The trap penalty (-2.0) provides a secondary, denser negative signal that tests whether each algorithm can learn from punishment without getting stuck in avoidance loops.

1.3 Research Questions

The environment was designed to investigate: (1) whether value-based methods (DQN) converge more reliably than policy-gradient methods (REINFORCE) under semi-sparse rewards with obstacles; (2) how sensitive each algorithm is to the choice of learning rate and discount factor; and (3) how seed-to-seed variability compares across the two algorithmic families.

1.4 Gymnax Implementation

The environment subclasses `gymnax.environments.environment.Environment` and uses Flax's `@struct.dataclass` for immutable state. This ensures full compatibility with JAX's functional paradigm: the `step_env` method is pure (no side-effects), enabling `jax.jit` compilation and `jax.lax.scan` for episode collection. All movement logic uses `jnp.where` for branchless execution on GPU.

2. JAX Implementations

2.1 Project Structure

The codebase is organised into a modular `src/` package with subdirectories for agents, environments, networks, training loops, replay buffer, evaluation, visualisation, and utilities. Configuration is centralised in a YAML file, and a shell script (`run_sweep.sh`) orchestrates the full hyperparameter sweep as individual Python processes to manage GPU memory.

```
assignment2/
    config.yaml
    run_sweep.sh
    run_dqn_single.py / run_reinforce_single.py /
    run_random_single.py
    src/
        agents/ — DQNAgent, PolicyAgent, RandomAgent
        envs/ — ObstacleTrapGridWorld (Gymnax)
        networks/ — q_network.py, policy_network.py (pure JAX)
        training/ — train_dqn.py, train_reinforce.py, rollout.py
        replay/ — replay_buffer.py
        evaluate/ — evaluate_dqn.py, evaluate_policy.py
        viz/ — viz_rl.py (matplotlib plotting)
        utils/ — reusable.py (Timer, RLMetricsDataset, GPU utils)
```

2.2 Neural Network Architecture

Both algorithms share the same 3-layer MLP backbone implemented in pure JAX (no Flax/Optax): input (2 neurons — row, col) → hidden layer (64 units, tanh) → hidden layer (64 units, tanh) → output. For DQN, the output layer has 4 neurons producing Q-values (one per action).

For REINFORCE, the output produces 4 logits passed through softmax to form a categorical policy distribution. Weights are initialised with a scale of 0.1 and biases are zeroed. The parameter dictionary is a plain Python dict of JAX arrays (W1, b1, W2, b2, W3, b3), making gradient computation and SGD updates straightforward via `jax.value_and_grad` and `jax.tree_util.tree_map`.

2.3 DQN Implementation

Episode collection is fully on-device using `jax.lax.scan`. Each episode runs for a fixed `max_steps=50` timesteps with a done-mask to zero-out post-terminal transitions, avoiding Python-level branching. Transitions are inserted into a circular replay buffer (capacity 2500) using `jax.lax.fori_loop` for batch insertion. The Q-network update is `@jax.jit`-compiled and performs 4 mini-batch SGD steps per episode (batch size 32), the SGD is updated only 4 times in episode in order to control the learning time of the agent, an ideal scenario would be to update the agent after every step it takes. A separate target network is synced every 100 global steps. Exploration follows a linear epsilon-greedy schedule decaying from 1.0 to
0.1 over the first 100 episodes.

2.4 REINFORCE Implementation

REINFORCE uses Monte Carlo policy gradient with return normalisation. Episode rollouts are collected via `jax.lax.scan` (same as DQN). Discounted returns are computed in reverse using `jax.lax.scan` — a fully on-device backward pass that avoids Python loops. The policy gradient loss is: L = -(1/T) sum_t [G_t * log pi(a_t|s_t)], computed over valid timesteps using a done-mask and `jax.vmap` for vectorisation. Returns are normalised (zero mean, unit variance) to reduce gradient variance. The update step is `@jax.jit`-compiled with manual SGD (no Optax).

2.5 Random Agent (Baseline)

The random baseline selects uniformly among the 4 actions using `jax.random.randint` — no learning, no parameters. It runs 100 episodes per seed across 10 seeds to establish a performance floor.

2.6 Key JAX Patterns

Several JAX-specific patterns were used throughout:

(i) `lax.scan` for episode collection eliminates Python while-loops and enables full GPU utilisation;
(ii) `lax.fori_loop` for batch replay-buffer insertion;
(iii) `jax.vmap` for batched Q-value computation and per-step REINFORCE loss;
(iv) explicit parameter dicts with `tree_map` for SGD — pure functional style without Optax;
(v) `jnp.where` for branchless environment logic;
(vi) JAX PRNG key splitting for reproducibility across seeds.
(vii) export XLA_PYTHON_CLIENT_PREALLOCATE=false
export XLA_PYTHON_CLIENT_MEM_FRACTION=0.5 This lines has helped in controlling the memory allocation with JAX on GPU. By default JAX reserves 75% of the GPU resources and does not release after each training session.

3. Experimental Setup

3.1 Hyperparameter Search

A full grid search was conducted over 3 learning rates (0.001, 0.01, 0.1) and 2 discount factors (gamma = 0.9, 0.99), yielding 6 configurations per algorithm. Each configuration was run with 10 independent seeds (0–9), totalling 60 DQN runs and 60 REINFORCE runs plus 10 random baseline runs. All agents trained for 250 episodes with max_steps=50 per episode.

Hyperparameter	Values
Learning rate	0.001, 0.01, 0.1
Discount factor (gamma)	0.9, 0.99
Seeds	0, 1, 2, ..., 9
Episodes	250
Max steps per episode	50
Hidden dimension	64
DQN replay buffer capacity	2500
DQN batch size	32
DQN epsilon decay	1.0 → 0.1 over 100 episodes
DQN target update freq	100 steps

Table 2: Hyperparameter search space.

3.2 GPU Setup

All experiments ran on a personal workstation with a CUDA-enabled GPU (detected as `cuda:0` via JAX). Environment variables `XLA_PYTHON_CLIENT_PREALLOCATE=false` and `XLA_PYTHON_CLIENT_MEM_FRACTION=0.5` were set to prevent JAX from pre-allocating 75% of VRAM, enabling sequential sweep execution. Total wall-clock time for the full sweep (130 runs) was approximately 10.5 hours (~5 min per run average). JIT warmup was performed before each run to exclude compilation overhead from timing.

Exact GPU Details:

GPU Name: NVIDIA RTX 5060 8GB
CUDA Version: 12.9

3.3 Evaluation Protocol

DQN was evaluated periodically (every 50 episodes) using greedy action selection over 25 episodes to compute success rate.

REINFORCE was evaluated at the final episode using greedy (argmax) action selection.

The final summary metrics (final_mean_reward, final_success_rate) were computed from the last evaluation of each run.

4. Results and Analysis

4.1 Learning Curves: Mean ± Standard Error

Figure 1 shows the mean reward over episodes for DQN and REINFORCE, averaged across all 60 runs (10 seeds × 6 HP configs) with standard error bands. Both algorithms start near -4.5 (similar to the random baseline) and improve over training. REINFORCE shows faster initial progress due to its direct policy optimisation, but DQN catches up and slightly surpasses it by episode 200 as Q-value estimates stabilise. The wide SE bands reflect high variance across hyperparameter configurations.

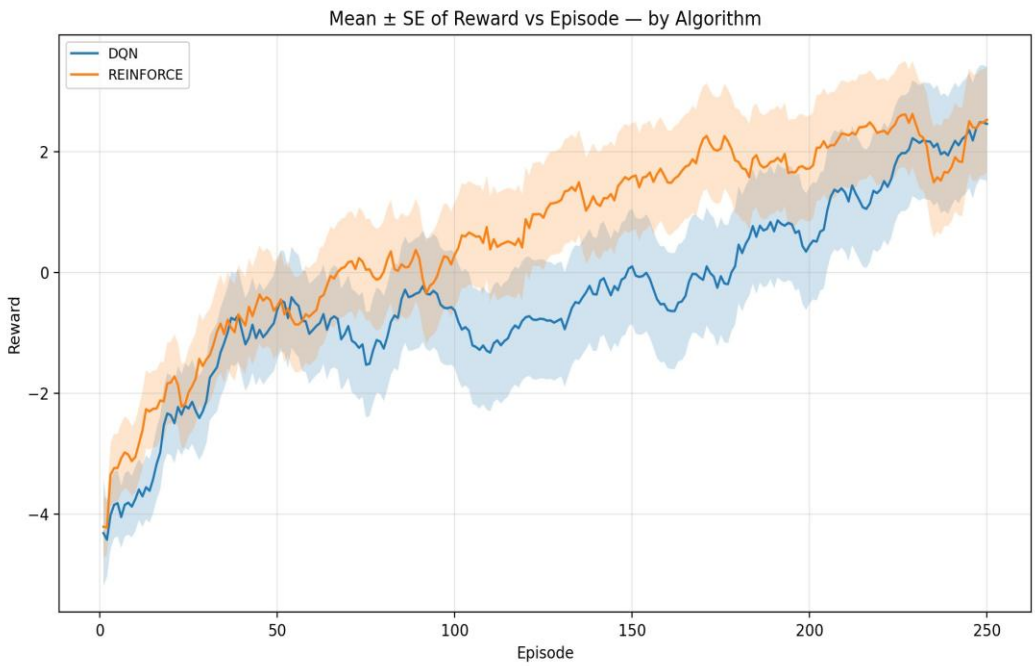

Figure 1: Mean ± SE of reward over episodes for DQN and REINFORCE (all HP configs, 10 seeds each).

4.2 Individual Seed Overlays

Figure 2 overlays all 60 individual DQN training curves (one per seed × HP config), revealing a strong bimodal pattern: some runs converge to high rewards (~+8) while others remain stuck near -5 throughout training. This bimodality is driven almost entirely by the learning rate — runs with lr=0.001 rarely converge in 250 episodes, while lr=0.01 converges most reliably.

The Images seems to be blurry but if we have a look at the dataset in the metrics/assignment2_metrics.csv files we can get the exact values.

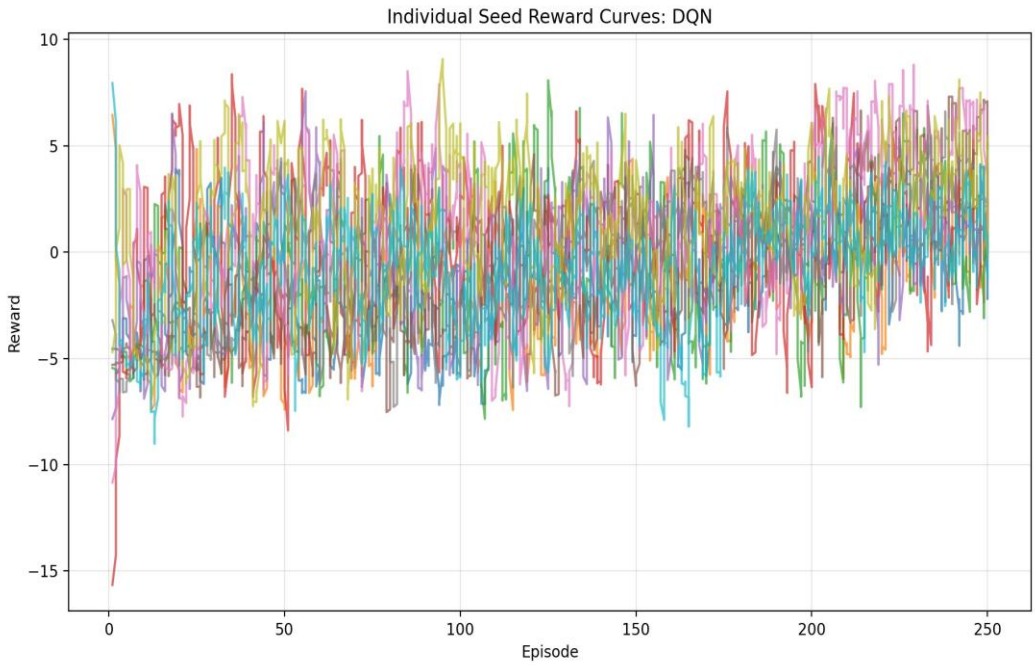

Figure 2: Individual seed reward curves for DQN (60 runs overlaid).

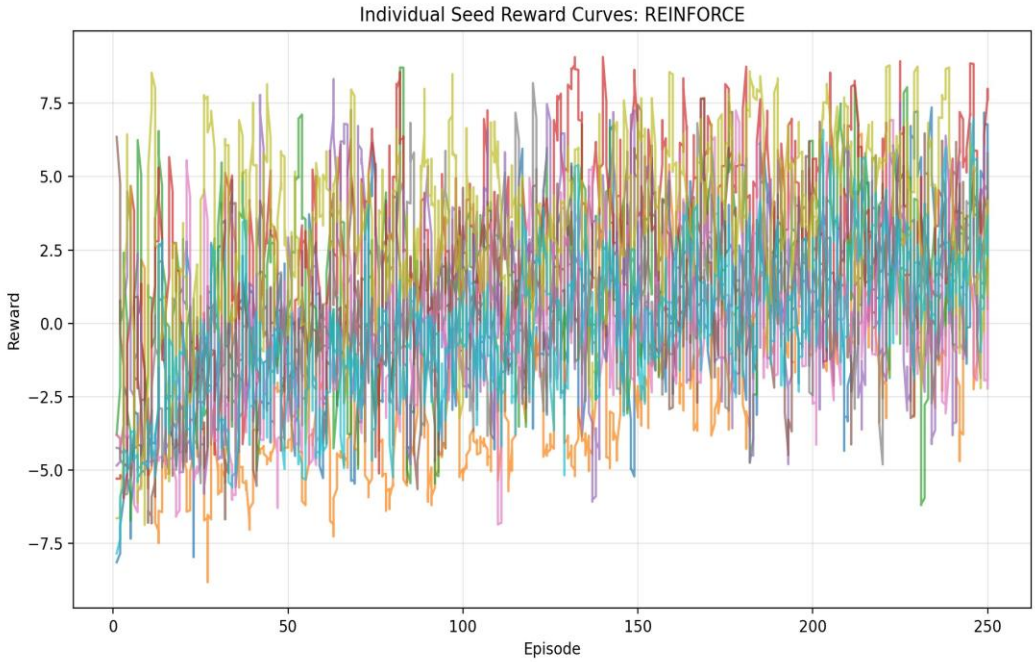

Figure 3: Individual seed reward curves for REINFORCE (60 runs overlaid).

4.3 Hyperparameter Sensitivity

Figure 4 displays a heatmap of final mean reward for each (learning rate, gamma) combination, averaged over 10 seeds. The key findings are:

DQN: The best configuration is lr=0.01, gamma=0.9 (mean reward 9.06, 100% success rate). A lower learning rate (0.001) is too slow to converge in 250 episodes, achieving only 20% success. A higher learning rate (0.1) works well with gamma=0.9 (80% success) but destabilises with gamma=0.99 (only 20% success) — suggesting that the combination of aggressive updates and far-sighted discounting causes Q-value oscillation.

REINFORCE: The best configuration is lr=0.1, gamma=0.99 (mean reward 5.66, 79% success). Unlike DQN, REINFORCE benefits from the highest learning rate across both gamma values. This is expected: REINFORCE's stochastic gradient estimates have high variance, so larger step sizes are needed to make progress before the signal is lost. Low learning rates (0.001) lead to near-random performance (~11% success), essentially failing to learn within the training budget.

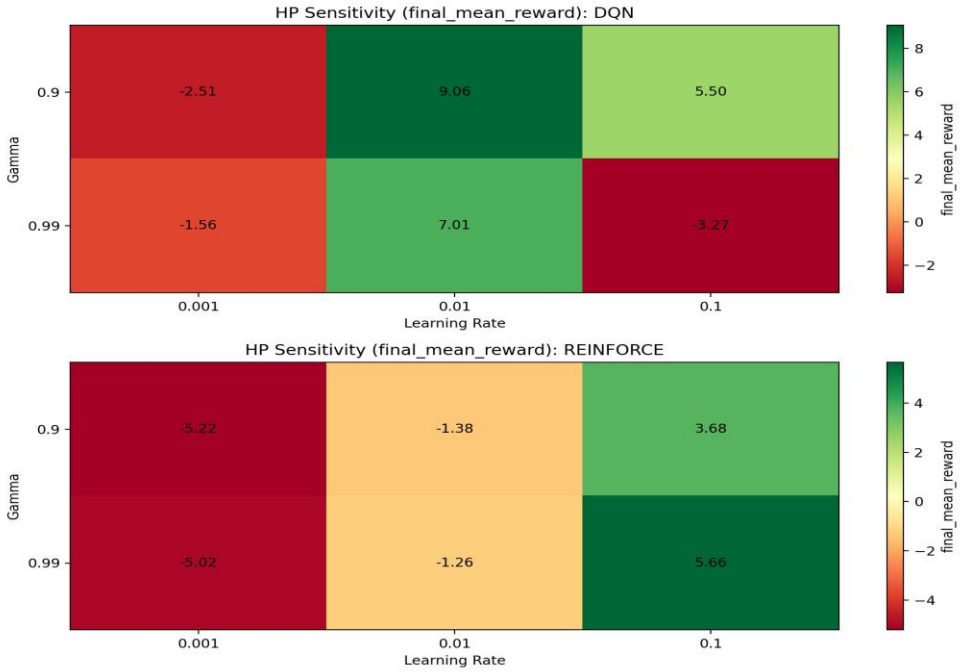

Figure 4: Hyperparameter sensitivity heatmap — final mean reward by (learning rate, gamma).

4.4 Summary Comparison

Algorithm	Best Config	Mean Reward	Success Rate	Std (Reward)
DQN	lr=0.01, γ=0.9	9.06	100%	0.94
REINFORCE	lr=0.1, γ=0.99	5.66	79%	6.94
Random	—	-3.50	16%	0.42

Table 3: Best hyperparameter configuration for each algorithm (averaged over 10 seeds).

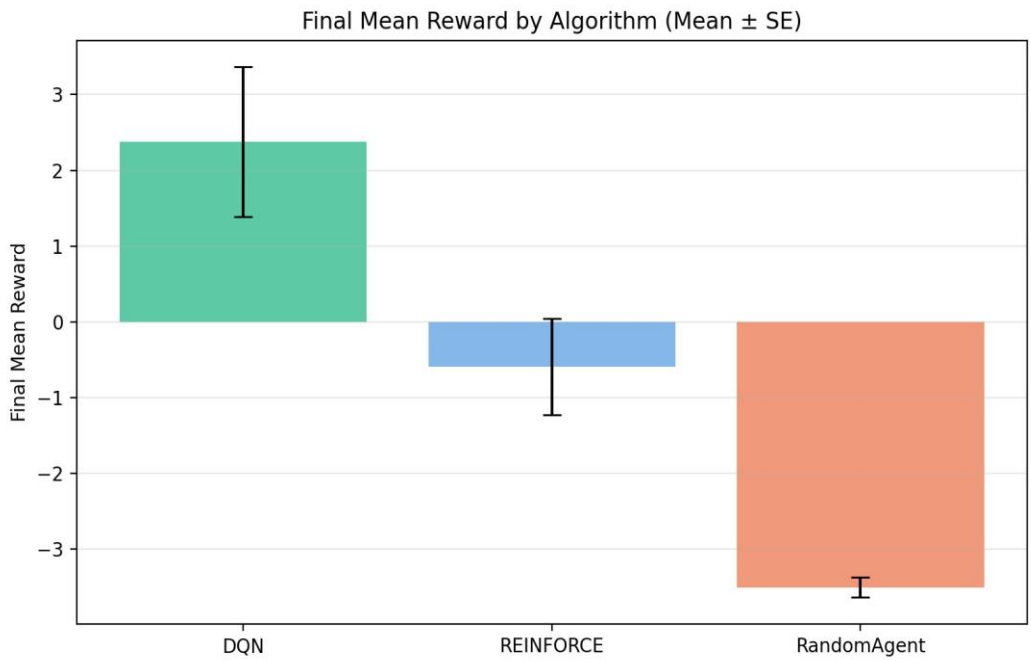

Figure 5: Final mean reward by algorithm (averaged across all HP configs and seeds).

5. Policy Visualisation

Figures 6 and 7 show the greedy policy (argmax action) learned by the best DQN and REINFORCE configurations respectively, displayed as directional arrows on the grid. Black cells are obstacles, pink is the trap (T), light blue is the start (S), and green is the goal (G).

DQN (lr=0.01, gamma=0.9) learns a clean, efficient policy: the agent moves right along the bottom row, then proceeds upward along the right side of the grid to reach the goal. Every cell shows a consistent, purposeful direction. The policy tries to avoids both the obstacles and the trap, choosing the longer but safer right-then-up route over the more direct (but trap-adjacent) diagonal but eventually goes inside the trap which could explain why the reward is less for evaluations.

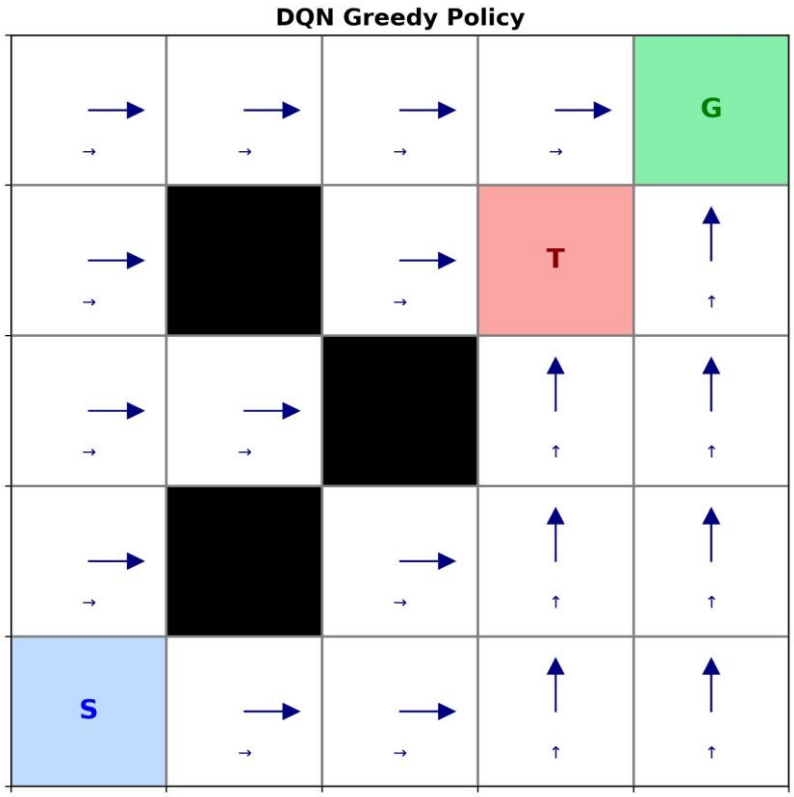

Figure 6: DQN best greedy policy (lr=0.01, gamma=0.9). Arrows show the argmax action at each cell.

REINFORCE (lr=0.1, gamma=0.99) also learns a viable policy but with a different routing preference: it moves predominantly upward first and then right along the top row. This path is also optimal (8 steps) but reflects the different optimisation trajectory of policy gradients. Some cells near the trap show the agent moving right into (1,4) then up to the goal — the policy successfully avoids the trap at (1,3) by routing through (1,2) rightward, bypassing the trap column.

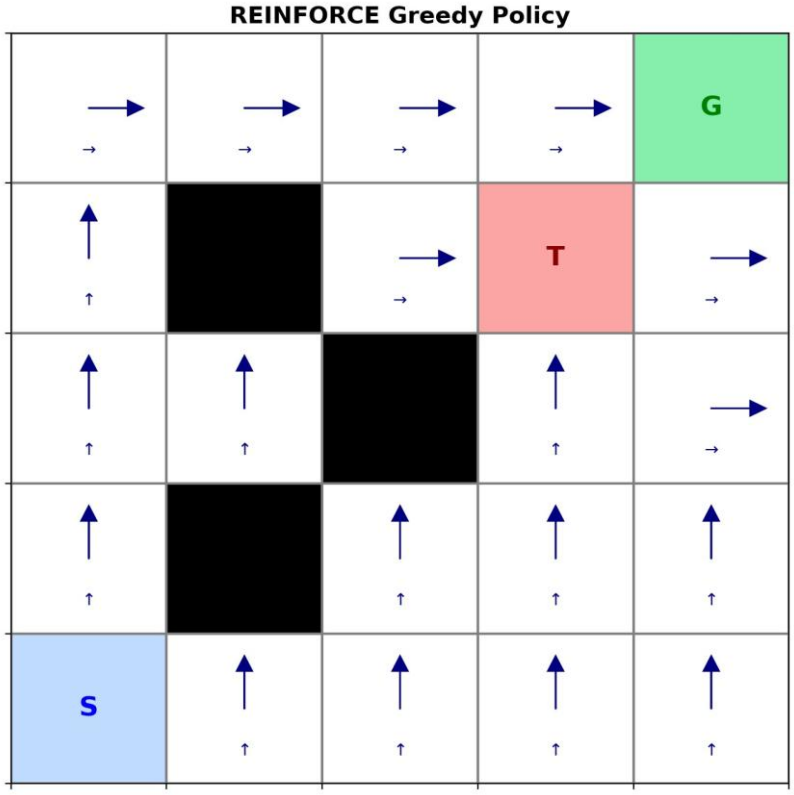

Figure 7: REINFORCE best greedy policy (lr=0.1, gamma=0.99). Arrows show the argmax action at each cell.

6. Reflection

6.1 Mean Performance vs. Seed Variability

DQN exhibits high cross-seed variability for suboptimal hyperparameter settings — its standard deviation ranges from 0.94 (at the best config) to 7.26 (at lr=0.1, gamma=0.9). This variance is largely driven by whether the epsilon-greedy exploration finds the goal early enough to bootstrap the Q-values. When it does, DQN converges rapidly and consistently; when it does not, the agent can get trapped in local minima. REINFORCE's variance is more uniformly high (6.2–7.5) across all configurations except the very low learning rate, suggesting that the high variance is intrinsic to the Monte Carlo gradient estimates rather than driven by exploration luck.

To assess whether 10 seeds are sufficient, one could compute confidence intervals on the mean or apply bootstrap resampling. The bimodal failure pattern in DQN suggests that reporting only the mean is misleading — a histogram of final rewards per seed would better characterise performance. Increasing the training budget (e.g. 1000 episodes) would likely reduce DQN's failure rate for moderate learning rates while having a smaller effect on REINFORCE, which requires more fundamental variance reduction techniques (e.g. baselines, entropy regularisation) to improve reliability.

6.2 JAX's Functional Style

JAX's functional paradigm strongly shaped the implementation. All environment state is immutable (Flax dataclasses), and transitions are computed as pure functions. This enables `lax.scan` for episode rollouts — a single fused GPU kernel replaces the standard Python for-loop. The replay buffer stores its pointer and size as JAX scalars, allowing `lax.fori_loop` to batch-insert transitions without CPU–GPU sync.

The main challenge was debugging: since `lax.scan` traces through the computation symbolically, standard print-debugging does not work. Errors in shape or type manifest as cryptic XLA compilation failures. The solution was to develop and test each component first in eager Python mode, then wrap in `jit`/`scan` once the logic was verified.

The lack of Optax was deliberate: using explicit `tree_map(lambda p, g: p - lr * g, ...)` for SGD made the gradient mechanics transparent and educational. In production code,
Optax would be preferred for more complex optimisers (Adam, gradient clipping), but for vanilla SGD in a small network, manual updates are equally correct and more instructive.

6.3 Environment Design and Research Questions

The semi-sparse reward structure (large goal bonus, small step penalty) was designed to stress-test credit assignment. The results confirm the hypothesis: DQN's ability to propagate value estimates backwards through the Bellman equation gives it a strong advantage over REINFORCE, which must rely on complete episode returns. The trap at (1,3) provided a useful secondary signal — both algorithms learned to avoid it, but DQN's Q-value decomposition allowed it to learn the trap's cost from even a single encounter, while REINFORCE needed multiple episodes to dilute the trap's effect on the return signal.

The obstacle placement along the diagonal was effective in preventing trivial solutions. Without obstacles, both algorithms would converge almost instantly; with them, the agent must learn a multi-step detour, creating enough complexity to reveal differences between the methods.

6.4 GPU Usage

All experiments ran on a personal GPU workstation using JAX's CUDA backend (`cuda:0`). Each of the 130 runs (60 DQN + 60 REINFORCE + 10 random) was launched as a separate Python process via the shell orchestrator (`run_sweep.sh`) to ensure clean GPU memory between runs. JIT warmup was performed before timing. Average per-run wall time was approximately 300 seconds (5 minutes), totalling ~10.5 hours for the full sweep.

The implementation part of model training was very demanding , the initial code written by me was not that able to complete the training with the available resources. The problems I have faced with different platforms are like this.

6.5 Limitations and Solutions:

1) Logging: Each agent is logged for the reward they have received in each episode this particular task is completed by CPU, this resulted changing of device from GPU and CPU consistently, I tried finding other options of logging with the GPU Device but was not able to.
2) Memory Allocation: The JAX is notorious for allocating 75% of GPU resources if not controlled by user. It took me majority of the time to debug this issue. This issue resulted in complete stalling of model training after 40%-50% of the hyperparam sweep.
3) Orchestrating: I initially tried running a main.py file which will orchestrate the hyperparameter sweep or grid search taking the hyperparams from config.yaml , but this resulted in hogging the System Memory and slowing down the whole system. So I tried implementing the grid search with the help of bash script of **run_sweep.sh** which independently calls each agent for training with each combination of hyperparameters and saves the output(model_checkpoints, episode_metrics, evaluation_metrics) of the agents and free ups the memory frequently.
4) Different Platforms:
    a. Google Colab Premium: The Colab was giving promising training times at the start but as we progressed with our training the system memory was becoming the issue, the metrics dataset was created from each episode of training and again evaluation dataset was also hogging the memory. I tried multiple times , eventually exhausting my 100 credits.
    b. Kaggle: Kaggle was a promising contender for me and I did use it for initial runs with 2xT4 GPUs. But session limitation of only 12hours the training was continuously getting cancelled after 12hours.
    c. Alliance Can: I do posses a compute resource with CCDB but the storage resources provided to me are not suitable for the storage requirements of the virtual environment(almost 10GB) of the assignment.
    d. Personal Workstation: By solving the issue of JAX memory preallocation and orchestrating issue of python files , I was able to efficiently carry out the whole training without exhausting the resources , one key insight I have observed during model training was this updated orchestrating method made my average resource allocation drop from 90% to only 20% , creating room for more multiple training batches if needed in the future cases.

7. AI Tool Usage

Claude (Anthropic) was used as a coding assistant through this project. Its contributions and their disposition are summarised below:

Accepted suggestions: Claude assisted primarily with **GPU memory management, dataset merging, and training orchestration**, helping diagnose and resolve memory allocation issues during batch processing, combining experimental results across multiple runs, and sweep execution.

Rejected / modified suggestions: (i) Claude initially suggested using Optax for optimisation, which was rejected in favour of manual SGD to keep the implementation transparent and educational; (ii) Some initial network architectures were too large (128/256 hidden units) for a 5×5 grid — these were scaled down to 64 units; (iii) Claude suggested using `vmap` for parallel seed execution, which was not adopted due to GPU memory constraints on the personal workstation.

Report writing: I used to do analysis on the output results of each agent , to understand the behaviour of each agent with the respective hyperparameter.